

Essential Commands and Workflows



Praveenkumar Bouna

www.CodeWithPraveen.com

Demo: Creating the Project from Scratch

Understanding Unfamiliar Codebases

The Onboarding Challenges:

- Steep learning curves
- Outdated documentation
- Unclear explanations
- Undocumented decisions

How Claude Code Helps:

- Interactive chat about codebase
- Explains in simple terms
- Identifies relationships
- Reveals architectural patterns

Types of Questions You Can Ask

Architecture Questions

"What's the overall architecture of this project?"
"How do the different modules interact?"

Functionality Questions

"How does the authentication flow work?"
"Where is the database connection configured?"

Dependency Questions

"What external libraries does this project use?"
"Why is library X included?"

Specific Function Questions

"What does the calculate_metrics function do?"
"What are the parameters for this API endpoint?"

Start broad, then narrow.

First: *“Give me the project structure and core components.”*

Next: *“Show me how requests flow from the router to the auth middleware.”*

Be specific.

Instead of: *"Tell me about this code"*

Try: *"How does the user authentication flow work in this app?"*

Demo: Exploring the Project Codebase

Demo: Debugging with Claude Code

Requesting Code Changes with Natural Language

Power of Natural Language Requests

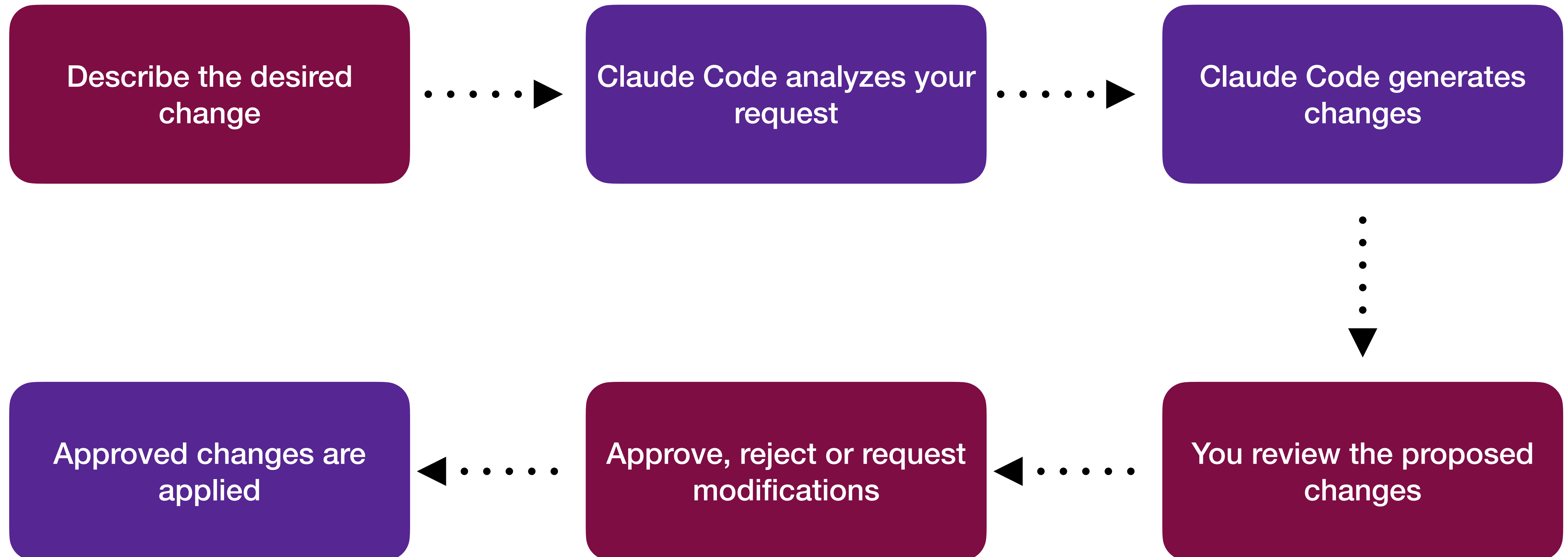
Describe what you want, not how to implement it

Claude Code generates the changes for you

No need to manually edit files

Review before applying

Workflow: How It Works



Claude Code Permission Modes

Permission Mode

It determines how Claude Code interacts with your codebase.

Types of Permission Modes

1. Plan Mode - Claude can analyze but not modify files or execute commands

```
> /clear  
└ (no content)
```

```
> include steps in readme.md to create python environment
```

```
|| plan mode on (shift+tab to cycle)
```

```
Thinking off (tab to toggle)
```

Types of Permission Modes

2. Accept Edits - Automatically accepts file edit permissions for the session

```
> /clear  
└ (no content)
```

```
> include steps in readme.md to create python environment
```

```
▶▶ accept edits on (shift+tab to cycle)
```


Types of Permission Modes

3. Standard behavior - Prompts for permission on first use of each tool (default)

```
> /clear  
└ (no content)
```

```
> include steps in readme.md to create python environment
```



Standard mode

Toggle by pressing `Shift + Tab` key.

```
> /clear  
└ (no content)
```

```
> include steps in readme.md to create python environment
```

```
▶▶ accept edits on (shift+tab to cycle)
```

Thinking Mode

It determines how Claude Code thinks about your requests.

Types of Thinking Modes

Thinking off: Claude Code does not think about your requests

```
> /clear  
  | (no content)
```

```
> include steps in readme.md to create python environment
```

```
  " plan mode on (shift+tab to cycle)
```

```
Thinking off (tab to toggle)
```

Types of Thinking Modes

Thinking on: Claude Code thinks about your requests before responding

```
> /clear  
  | (no content)
```

```
> include steps in readme.md to create python environment
```

```
» plan mode on (shift+tab to cycle)
```

```
Thinking off (tab to toggle)
```

Toggle by pressing `Tab` key.

```
> /clear  
└ (no content)
```

```
> include steps in readme.md to create python environment
```

```
■ plan mode on (shift+tab to cycle)
```

```
Thinking off (tab to toggle)
```


Setting the Default Behavior

~/.claude/settings.json

```
{  
  "permissions": {  
    "defaultMode": "plan"  
  },  
  "alwaysThinkingEnabled": true  
}
```

Demo: Making Changes with Approval

Demo: Git Integration and Version Control

Demo: Adding Documentation and CLAUDE.md Files

Thank you!



Praveenkumar Bouna

www.CodeWithPraveen.com